AI ASSISTANT CODING
ASSIGNMENT-15

NAME: K.MadhuPriya

HTNO: 2303A51583

BATCH:22

Task 1 – Student Attendance API

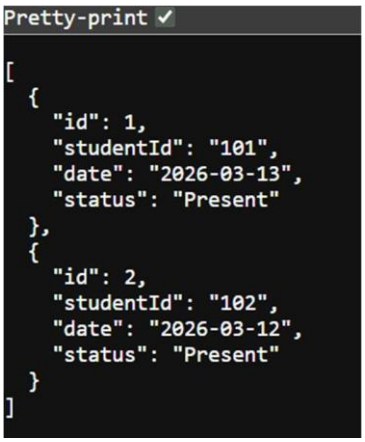
Task:
Create a RESTful API for student attendance tracking.

Instructions:
• Endpoints required:
o GET /attendance → View attendance records
o POST /attendance → Mark attendance
o PUT /attendance/{id} → Update attendance record
o DELETE /attendance/{id} → Remove attendance entry
• Ensure JSON-based request and response handling.

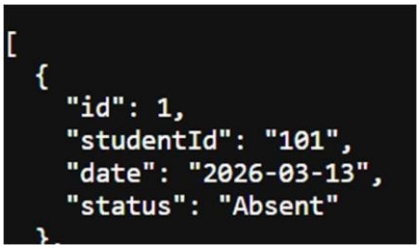
Expected Output:
• API for maintaining and updating attendance records.

GET:

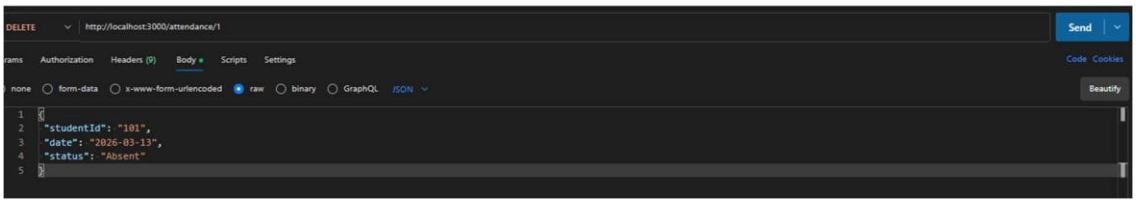
POST:

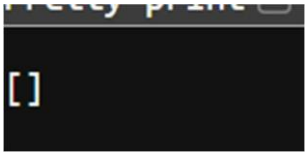


PUT:



DELETE:





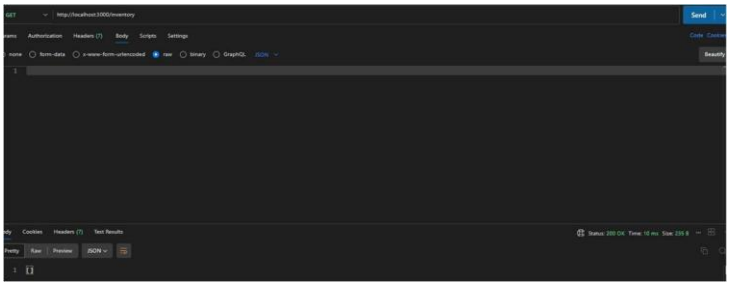
Task 2 – Inventory Management API

Task:
Develop a REST API for warehouse inventory management.

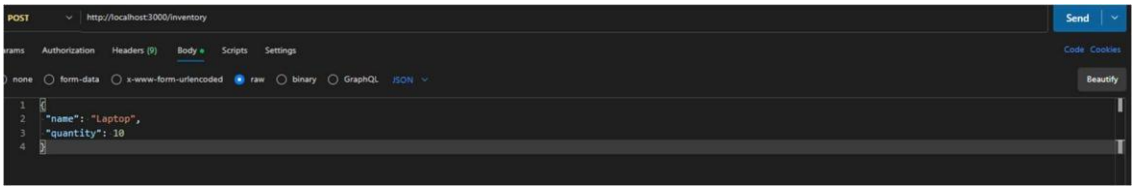
Instructions:
• Endpoints required:
o GET /inventory → List all items
o POST /inventory → Add new item
o PUT /inventory/{id} → Update stock quantity
o DELETE /inventory/{id} → Remove an item
• Implement validation for stock values.

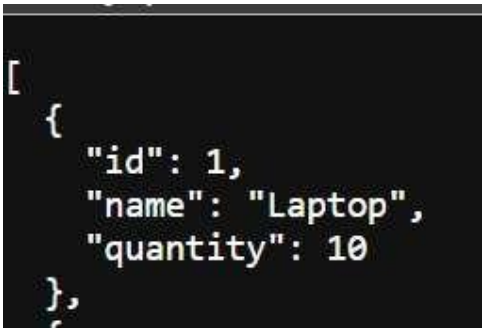
Expected Output:
• API capable of managing inventory stock levels.

GET:

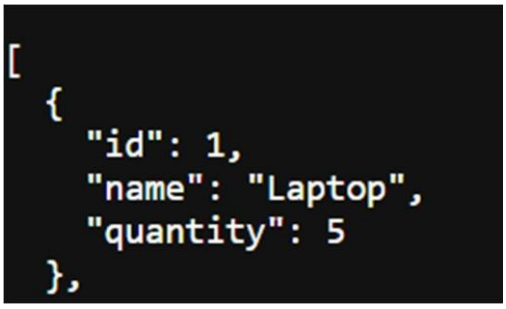


POST:

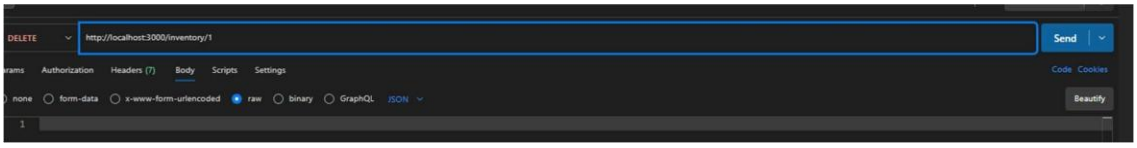




PUT:



DELETE:



Task 3 – Hotel Room Booking API

Task:
Develop a RESTful API for hotel room booking.

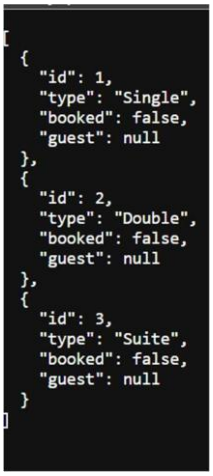
Instructions:

- Endpoints required:
  - o GET /rooms → View available rooms
  - o POST /rooms/book → Book a room
  - o GET /rooms/{id} → View booking details
  - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

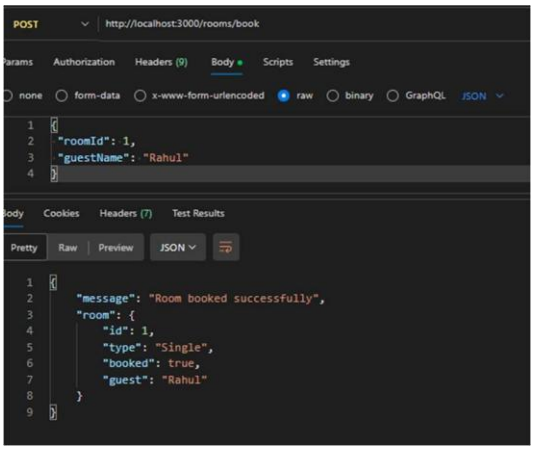
Expected Output:

- API simulating hotel room booking operations.

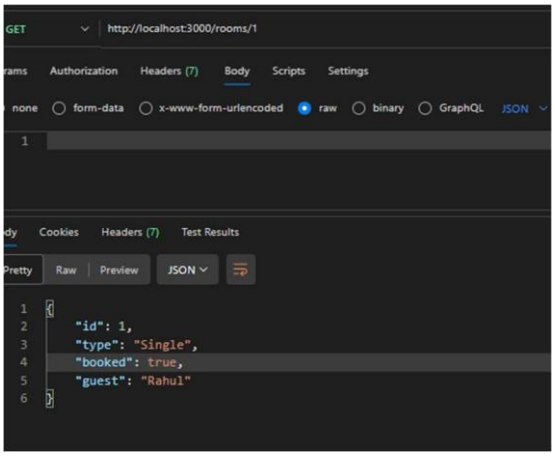
GET:



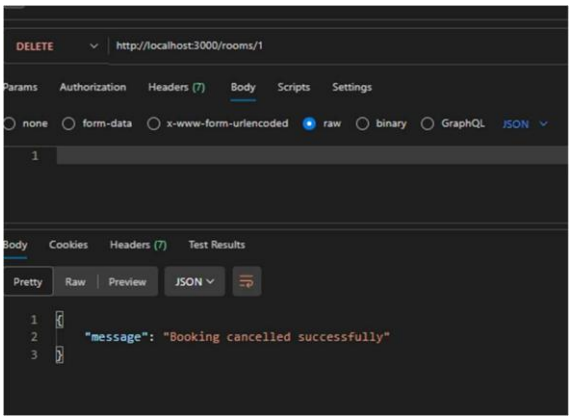
POST:



GET:



DELETE:



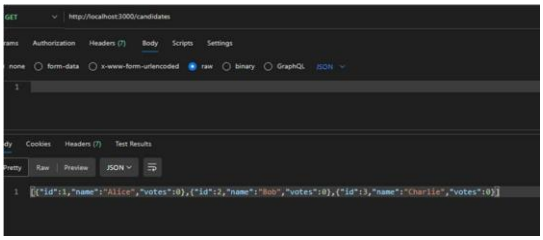
Task 4 – Online Voting API

Task:
Create a RESTful API for an online voting system.

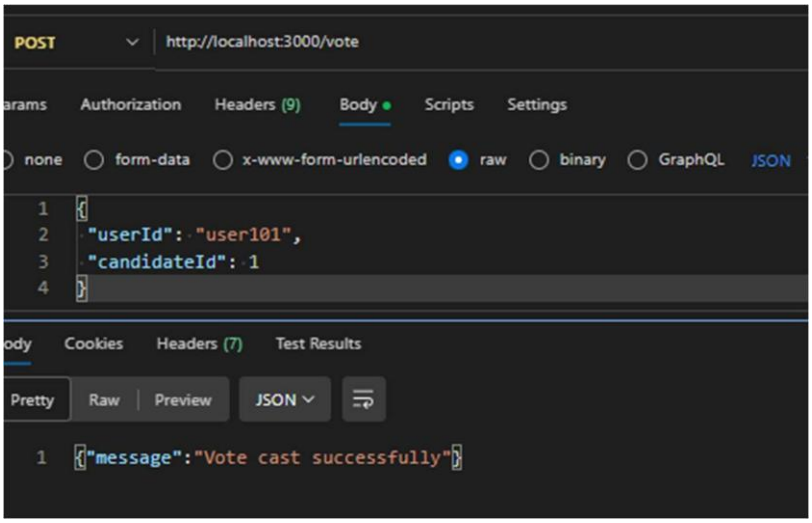
Instructions:
• Endpoints required:
o GET /candidates → List candidates
o POST /vote → Cast a vote
o GET /results → View voting results
• Ensure one vote per user using in-memory validation.

Expected Output:
• API simulating a secure online voting process.

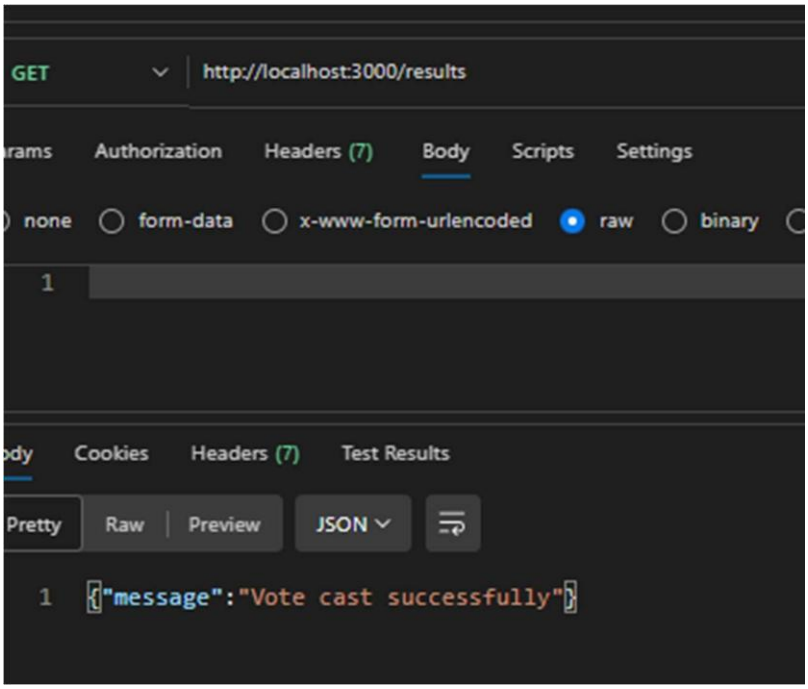
GET:



POST:



GET:



Task 5 – Teacher Management API

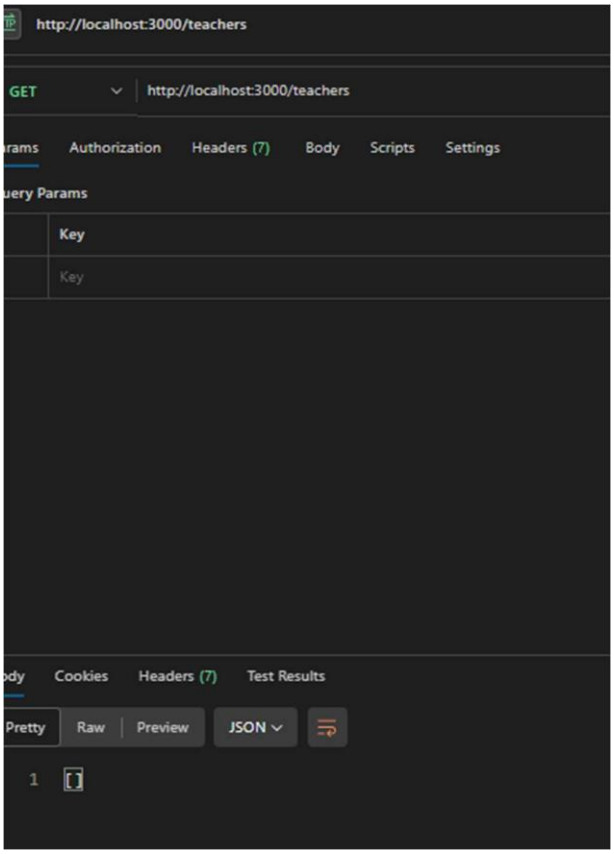
Task:
Develop a RESTful API to manage teacher records.

Instructions:
• Endpoints required:
o GET /teachers → List all teachers
o POST /teachers → Add a new teacher
o PUT /teachers/{id} → Update teacher details
o DELETE /teachers/{id} → Remove a teacher
• Ensure proper error handling and JSON responses.

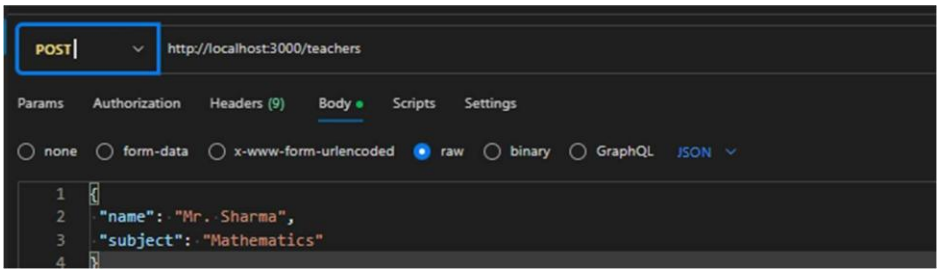
Expected Output:

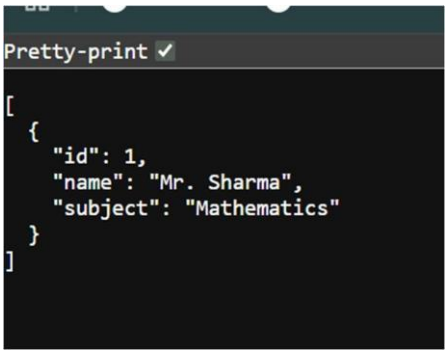
• Functional API for managing teacher information.

GET:

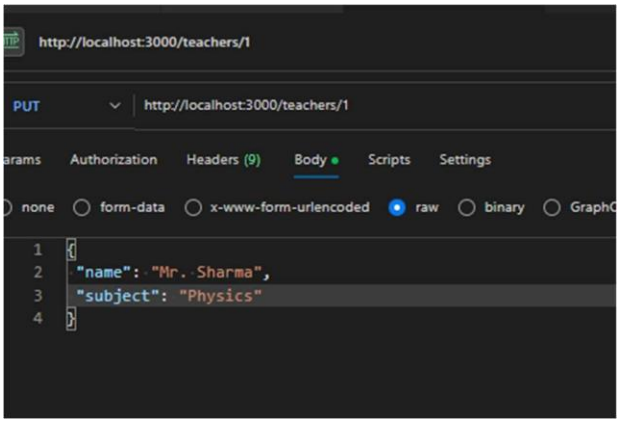


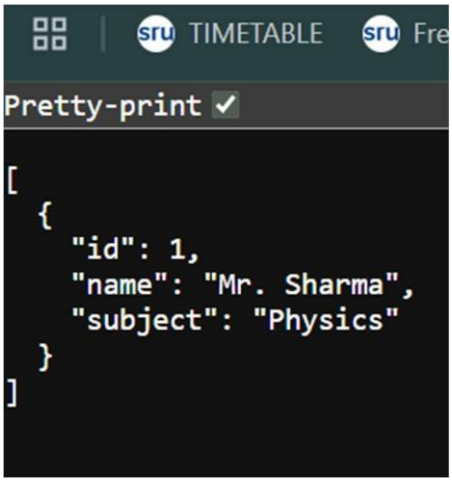
POST:



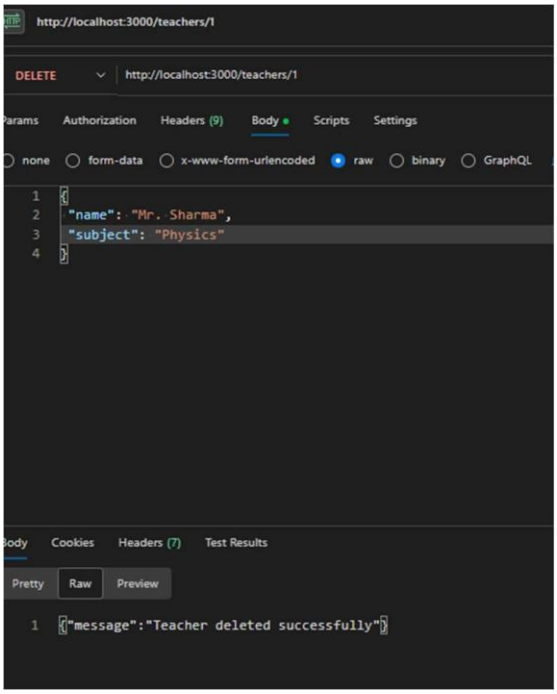


PUT:


http://localhost:3000/teachers/1
PUT
http://localhost:3000/teachers/1
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
{
"name": "Mr. Sharma",
"subject": "Physics"
}


TIMETABLE
Pretty-print
[
{
"id": 1,
"name": "Mr. Sharma",
"subject": "Physics"
}
]

DELETE:


http://localhost:3000/teachers/1
DELETE
http://localhost:3000/teachers/1
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
{
"name": "Mr. Sharma",
"subject": "Physics"
}
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
{"message":"Teacher deleted successfully"}